



MOBILNE APLIKACIJE

Vežbe 8

Lociranje i mape

2025/2026

Sadržaj

1. Uvod	3
2. Implementacija korišćenjem Google Maps API-ja	4
4. Implementacija korišćenjem OSMDroid-a	10
4.1. Prikaz trenutne lokacije	10
4.2. Crtanje ruta	13
5. Primer	16

1. Uvod

GPS je globalni navigacioni satelitski sistem koji omogućava određivanje lokacije i tačnog vremena. Sastoji se iz svemirskog, kontrolnog i korisničkog segmenta. Svemirski segment čine minimalno 24 satelita koji šalju signal na osnovu koga može da se odredi udaljenost od satelita i tačno vreme. Korisnički segment čine GPS prijemnici, koji primaju taj signal i metodom trilateracije izračunavaju svoju lokaciju i tačno vreme.

LocationManager

Klase *LocationManager* omogućava pristup sistemskom servisu za lociranje. To je glavna klasa Android Location API-a koja omogućava komponentama aplikacije da:

- periodično primaju informacije o lokaciji i orijentaciji uređaja i
- primaju obaveštenja kada se uređaj nađe u blizini zadate lokacije.

Lokacija uređaja može da se odredi na 3 načina:

- **GPS_PROVIDER** (GPS)
- **NETWORK_PROVIDER** (GSM, Wi-Fi)
- **PASSIVE_PROVIDER** (informacije o lokaciji koje je dobavila druga aplikacija)

LocationListener

Interfejs *LocationListener* prima obaveštenja od *LocationManager*-a kada se lokacija promeni.

U nastavku su data dva primera implementacije korišćenjem Google Maps API-ja i OSMDroid biblioteke.

2. Implementacija korišćenjem Google Maps API-ja

Mapa je umanjena slika zemljine površine na kojoj su prikazani raspored i uzajamna povezanost prirodnih, veštačkih i društvenih pojava.

Google Maps API omogućava korišćenje Google Maps servisa na Android platformi:

- prikaz mape
- rukovanje kamerom (pomeranje, zumiranje mape, itd.)
- postavljanje oznaka, linija, poligona, itd.

Da bi se koristile Google mape potrebno je izvršiti sledeće korake:

1. Preuzeti Google Play Service SDK
2. Preuzeti ključ za korišćenje Google Maps API-a
3. Uneti ključ u *AndroidManifest* datoteku
4. Zatražiti odgovarajuća prava pristupa u *AndroidManifest* datoteci
5. Deklarisati mapu kao pogled u odgovarajućem rasporedu
6. Definisati fragment/aktivnost koja prikazuje raspored

Prvi korak smo već izvršili prilikom inicijalnog konfigurisanja Android Studio-a. Potrebno je da imamo emulirani uređaj koji u sebi ima Google Play Service SDK.

Da biste dobili ključ potrebno je da se registrujete na *Google Cloud Platform Console* i da ostavite kreditnu karticu, ali se sam API ključ ne plaća. Za potrebe primera, omogućiti *Maps SDK for Android*, *Routes API* i *Directions API* u sekciji *APIS & Services*.

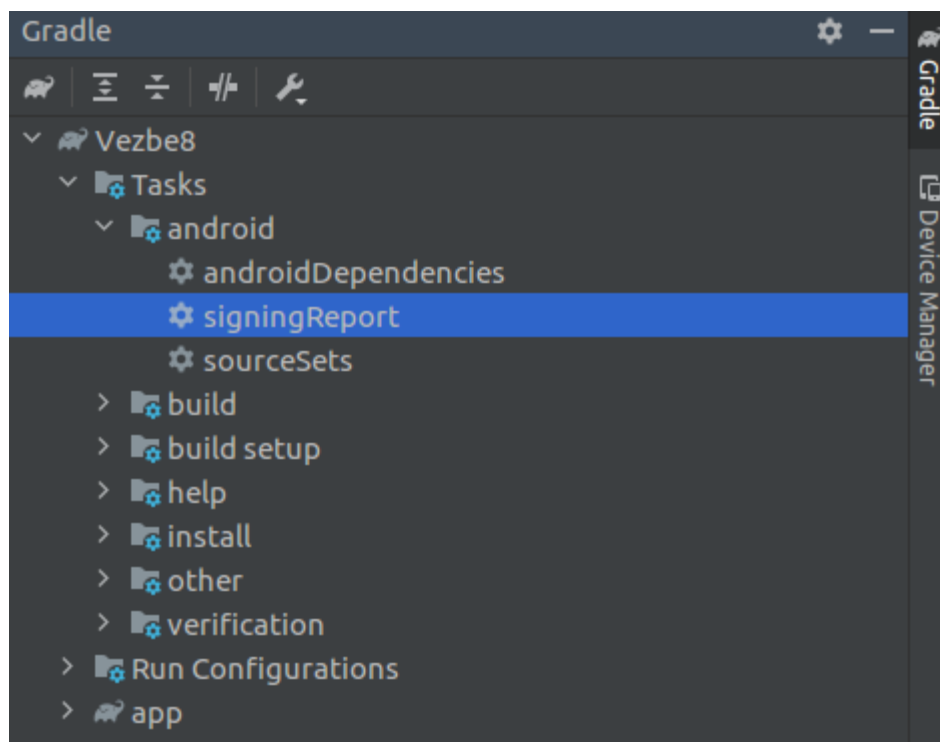
Detalje oko preuzimanja ključa možete pronaći na sledećem linku: <https://developers.google.com/maps/documentation/android-sdk/get-api-key>

Na istom linku nalazi se i uputstvo postavljanja restrikcija nad ključem. Prilikom postavljanja restrikcije, možete uneti imena paketa i SHA-1 sertifikata. Ime paketa predstavlja *applicationId* koji se nalazi u *build.gradle* datoteci na nivou modula (Slika 1).

```
android {  
    namespace 'com.example.vezbe8'  
    compileSdk 32
```

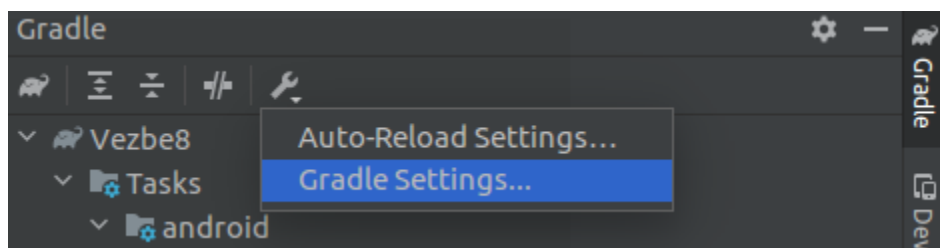
Slika 1. Ime paketa

Klikom na *gradle* paletu u gornjem desnom uglu Android Studio-a otvoriće se prozor sa strukturom projekta kao na slici 2. Dvoklikom na *signingReport* dobićemo ispis SHA-1 sertifikata u konzoli.



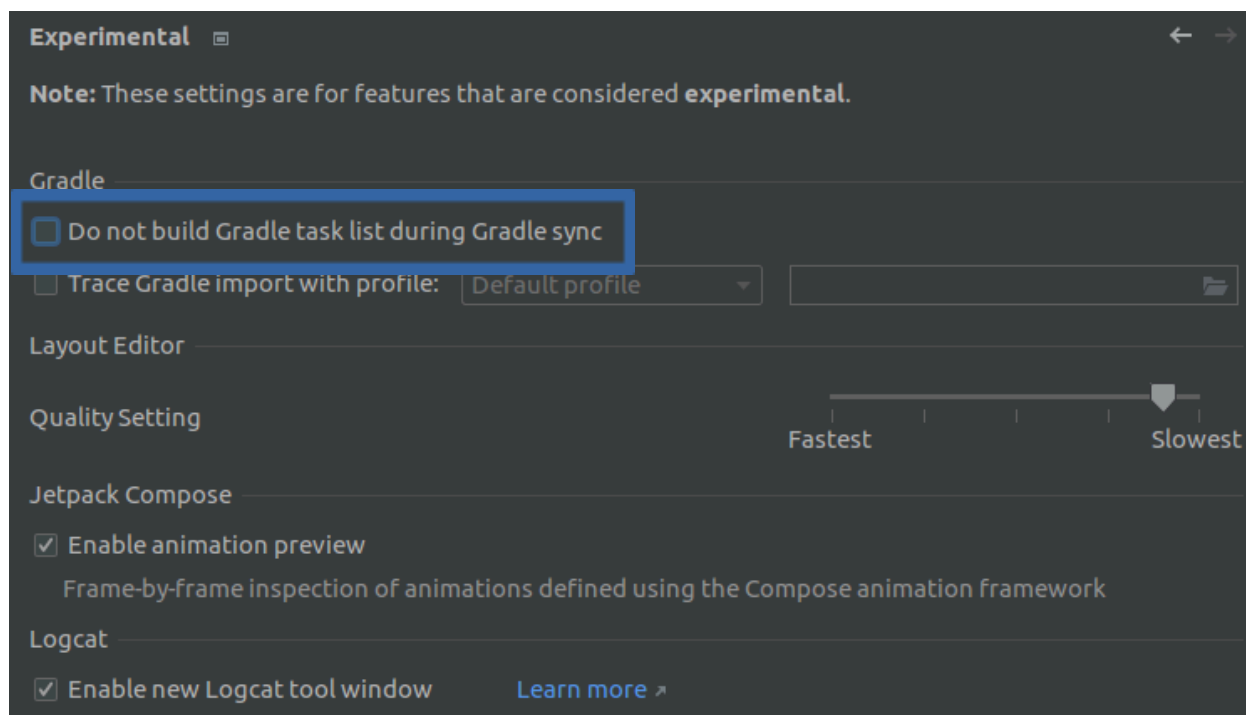
Slika 2. Pokretanje *signingReport*-a za dobijanje sertifikata

Ukoliko ne postoji folder *Tasks* kao na slici 2, potrebno je kliknuti na poslednju stavku menija u *gradle* paleti i izabrati *Gradle Settings...* (Slika 3).



Slika 3. *Gradle Settings*

U *Experimental* odeljku isključiti *Do not build Gradle task list during Gradle sync* opciju (Slika 4). Nakon bildovanja projekta, pojaviće se struktura kao sa slike 2.



Slika 4. *Experimental* tab *Gradle* podešavanja

U *AndroidManifest* datoteci naveli smo dva element `<meta-data>` (slika 5). Prvi element definiše verziju *Google Play* servisa, a u drugi element se unosi ključ.

```
<meta-data
    android:name="com.google.android.gms.version"
    android:value="@integer/google_play_services_version" />
<meta-data
    android:name="com.google.android.maps.v2.API_KEY"
    android:value="YOUR_KEY_GOES_HERE" />
```

Slika 5. Definisanje ključa u *AndroidManifest* datoteci

Umesto unošenja ključa direktno u *AndroidManifest* datoteku, preporučuje se čuvanje ključa u *local.properties* datoteci, nakon čega je potrebno iz *AndroidManifest* datoteke referencirati sačuvani ključ. Za podešavanje ovog dela potrebno je ispratiti korake 3 i 4 sa sledećeg linka: <https://developers.google.com/maps/documentation/android-sdk/config>

Preostali koraci, koji podrazumevaju kreiranje aktivnosti i omogućavanja odgovarajućih prava pristupa, se nalaze u nastavku ovog poglavlja.

Prvo smo u *AndroidManifest* datoteci dodali potrebna prava pristupa. Na slici 6 se nalazi primer definisanja statičkih prava pristupa. A u okviru *MapFragment-a* pozivaju se i dinamička prava pristupa.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      package="rs.reviewer">
4
5      <uses-permission android:name="android.permission.INTERNET" />
6      <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
7      <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
8      <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
9      <uses-permission android:name="com.google.android.providers.gsf.permission.READ_GSERVICES" />
10     <uses-permission android:name="android.permission.CHANGE_WIFI_STATE" />
11
12     <permission
13         android:name="rs.reviewer.MAPS_RECEIVE"
14         android:protectionLevel="signature" />

```

Slika 6. Definisanje statičkih prava pristupa

Kreirana klasa *MapFragment*, osim što nasleđuje *Fragment* implementira i interfejs ***OnMapReadyCallback*** koji sadrži metode za rad sa mapom. Ovaj fragment će služiti za prikaz i učitavanje mape. Korisniku se prikazuje trenutna lokacija, a omogućeno je i postavljanje novih markera na samoj mapi.

Unutar *onCreate()* metode neophodno je instancirati *FusedLocationProviderClient*. Ova klasa poseduje metode za pristup lokaciji iz različitih izvora (GPS, Wi-Fi...) i definisanje parametara za vreme i preciznost ažuriranja. Na slici 7 prikazano je instanciranje ove klase zajedno sa implementacijom provere dozvola za lokaciju.

```

FusedLocationClient = LocationServices.getFusedLocationProviderClient(requireActivity());

//...
requestPermissionLauncher = registerForActivityResult(
    new ActivityResultContracts.RequestMultiplePermissions(),
    permissions -> {
        Boolean fineLocationGranted = permissions.getOrDefault(Manifest.permission.ACCESS_FINE_LOCATION, false);
        Boolean coarseLocationGranted = permissions.getOrDefault(Manifest.permission.ACCESS_COARSE_LOCATION, false);
        if (fineLocationGranted != null && fineLocationGranted) {
            enableUserLocation();
        } else if (coarseLocationGranted != null && coarseLocationGranted) {
            enableUserLocation();
        } else {
            Toast.makeText(requireContext(), "Location permissions are required for map functionality.", Toast.LENGTH_LONG).show();
            requireActivity().getSupportFragmentManager().popBackStack();
        }
    }
);

```

Slika 7. Instanciranje *FusedLocationProviderClient*-a i provera dozvola

U metodi *onResume()*, pre nego što se prikaže mapa, proverava se da li je lokacija omogućena. Ukoliko nije, prikazuje se dijalog sa opcijom da korisnik omogući lokaciju koji vodi na stranicu za podešavanja (slika 8).

```
private void showEnableLocationDialog() {
    new AlertDialog.Builder(requireContext())
        .setTitle("Enable Location") Builder
        .setMessage("Location services are disabled. Please enable them in your settings to use the map.")
        .setPositiveButton(text: "Settings", (dialog, which) -> {
            Intent intent = new Intent(android.provider.Settings.ACTION_LOCATION_SOURCE_SETTINGS);
            startActivity(intent);
        })
        .setNegativeButton(text: "Cancel", (dialog, which) -> dialog.dismiss())
        .create() AlertDialog
        .show();
}
```

Slika 8. Prikaz dijaloga za omogućavanje lokacije

Ako su dozvole za pristup lokaciji dodeljene, mapa se inicijalizuje u metodi *onMapReady()*, gde se omogućava praćenje trenutne lokacije korisnika pozivanjem *enableUserLocation()* metode.

setMyLocationEnabled(true) omogućava mapi da prikazuje trenutnu lokaciju korisnika, dok se *LocationRequest* koristi za definisanje parametara za dobijanje ažuriranja lokacije, uključujući preciznost (*Priority.PRIORITY_HIGH_ACCURACY*) i interval ažuriranja. Implementacija je prikazana na slici 9.

```
map.setMyLocationEnabled(true);

// Definisanje parametara za dobijanje poslednje poznate pozicije
LocationRequest locationRequest = new LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, intervalMillis: 10000)
    .setMinUpdateIntervalMillis(5000)
    .build();

locationCallback = new LocationCallback() {
    no usages
    @Override
    public void onLocationResult(@NonNull LocationResult locationResult) {
        if (locationResult.getLastLocation() == null) {
            return;
        }
        Location location = locationResult.getLastLocation();
        updateMarker(location);
    }
};

fusedLocationClient.requestLocationUpdates(locationRequest, locationCallback, looper: null);
```

Slika 9. Implementacija dobavljanja trenutne lokacije

Nakon dobavljanja lokacije, pozivanjem *updateMarker()* metode vrši se ažuriranje iscrtavanja markera na mapi (slika 10). Mapa se zumira na novopostavljeni marker.


```

private void updateMarker(Location location) {
    LatLng currentLocation = new LatLng(location.getLatitude(), location.getLongitude());

    if (homeMarker != null) {
        homeMarker.remove();
    }

    homeMarker = map.addMarker(new MarkerOptions()
        .title("Your Position")
        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_ORANGE))
        .position(currentLocation));

    CameraPosition cameraPosition = new CameraPosition.Builder()
        .target(currentLocation).zoom(15).build();

    map.animateCamera(CameraUpdateFactory.newCameraPosition(cameraPosition));
}

```

Slika 10. Ažuriranje markera na mapi

Korisniku je omogućeno i postavljanje novih markera klikom na željenu poziciju. Ovo se postiže implementacijom listenera nad mapom (slika 11).

```

// Postavljanje klik listenera na mapu. Omogućava da se klikom na željenu poziciju postavi marker na mapi.
map.setOnMapClickListener(latLng -> {
    map.addMarker(new MarkerOptions()
        .title("YOUR_POSITION")
        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_AZURE))
        .position(latLng));

    CameraPosition cameraPosition = new CameraPosition.Builder()
        .target(latLng).zoom(14).build();

    map.animateCamera(CameraUpdateFactory.newCameraPosition(cameraPosition));
});

```

Slika 11. Implementacija listenera za postavljanje novih markera na mapi

Na samom kraju, definišemo metodu fragmenta *onPause()*, jer rad sa lokacijama dosta troši bateriju, pa čim završimo rad sa mapama, treba ukloniti klijenta za lokaciju (slika 12).

```

public void onPause() {
    super.onPause();
    // Prilikom pauziranja aktivnosti, potrebno je ukloniti klijenta za lokaciju.
    if (fusedLocationClient != null && locationCallback != null) {
        fusedLocationClient.removeLocationUpdates(locationCallback);
    }
}
}

```

Slika 12. Metoda *onPause*

4. Implementacija korišćenjem *OSMDroid*-a

OSMDroid predstavlja biblioteku koja omogućava rad sa *OpenStreetMap* podacima unutar Android aplikacija. Pruža osnovnu podršku za prikaz i manipulisanje mapom poput zumiranja, prikaza trenutne lokacije, postavljanje markera, crtanja ruta, kao i razna dodatna podešavanja.

Za implementaciju mapa pomoću ove biblioteke neophodno je navesti biblioteke unutar *build.gradle* datoteke (slika 13).

```

implementation libs.osmdroid.android
implementation libs.osmdroid.geopackage
implementation libs.okhttp
implementation libs.gson

```

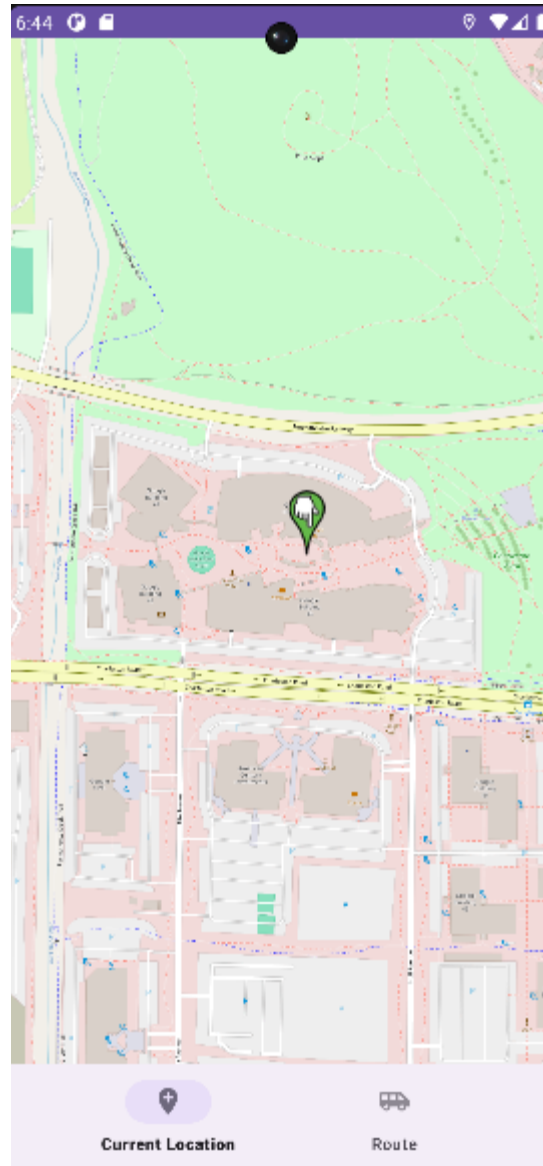
Slika 13. Potrebne biblioteke za implementaciju mapa

OSMDroid biblioteka se oslanja na druge *API*-je za dekodiranje adresa i proračunavanje ruta, pa je potrebno koristiti i *OkHttp* klijent za slanje i preuzimanje zahteva, kao i *gson* biblioteku za rad sa *json* podacima dobijenim putem ovih zahteva.

U nastavku biće detaljno objašnjen prikaz trenutne lokacije i iscrtavanje rute na osnovu unosa početne i krajnje lokacije. Primer aplikacije se sastoji iz dva fragmenta: *CurrentLocationFragment* i *RouteFragment*.

4.1. Prikaz trenutne lokacije

CurrentLocationFragment učitava mapu i postavlja marker na trenutnu poziciju uređaja (slika 14).



Slika 14. Prikaz trenutne lokacije sa markerom

Da bi prikazali mapu, koristimo objekat *MapView* koji nam je dostupan iz *OSMDroid* biblioteke. Za rad sa lokacijom koristimo *LocationManager*, dok sve promene lokacije pratimo putem *LocationListener*-a. Na slici 15. prikazana je inicijalizacija ovih elemenata.

```

public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
    View view = inflater.inflate(R.layout.fragment_current_location, container, attachToRoot: false);

    // Inicijalizacija OSMDroid-a. Postavljamo prikaz mape unutar layout-a fragmenta i omogućavamo multitouch.
    Configuration.getInstance().setUserAgentValue(requireContext().getPackageName());
    mapView = view.findViewById(R.id.map);
    mapView.setMultiTouchControls(true);

    // Inicijalizacija lokacionog menagera putem kog ćemo dobiti informacije o lokaciji
    locationManager = (LocationManager) requireActivity().getSystemService(Context.LOCATION_SERVICE);

    // Postavljanje listener-a za praćenje promena lokacije
    locationManager = new LocationListener() {
        @Override
        public void onLocationChanged(@NonNull Location location) {
            updateMapLocation(location);
        }
    };
    requestLocation();
    return view;
}

```

Slika 15. Inicijalizacija *OSMDroid*a i *LocationManager*-a

Nakon svake registrovane promene lokacije, poziva se metoda *updateMapLocation()* koja osvežava pogled mape i ažurira marker (slika 16).

```

private void updateMapLocation(Location location) {
    // Dobavljanje trenutne lokacije na osnovu geografske širine i dužine
    GeoPoint currentLocation = new GeoPoint(location.getLatitude(), location.getLongitude());

    // Inicijalizacija kontrolera mape koji služi za manipulisanje mapom (zum, boje, iscrtavanja...)
    // Mapa se centrira na prethodno dobavljenu poziciju
    IMapController mapController = mapView.getController();
    mapController.setZoom(18.0);
    mapController.setCenter(currentLocation);

    // Iscrtavanje markera na mapi korišćenjem prethodno dobavljene pozicije
    Marker marker = new Marker(mapView);
    marker.setPosition(currentLocation);
    marker.setTitle("You are here");
    mapView.getOverlays().add(marker);

    // Ažuriranje prikaza mape
    mapView.invalidate();
}

```

Slika 16. Prikaz markera i ažuriranje pozicije na mapi

Potrebno je voditi računa o datim dozvolama i pre svakog ažuriranja mape proveriti da li i dalje važe sve dozvole (slika 17). Nakon uspešne provere dozvola, moguće je preuzeti novu vrednost lokacije.

```

private void requestLocation() {
    if (ActivityCompat.checkSelfPermission(requireContext(), Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED &&
        ActivityCompat.checkSelfPermission(requireContext(), Manifest.permission.ACCESS_COARSE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(requireActivity(),
            new String[]{Manifest.permission.ACCESS_FINE_LOCATION, Manifest.permission.ACCESS_COARSE_LOCATION},
            PERMISSION_CODE);
        return;
    }
    // Traženje nove lokacije preko GPS provajdera
    // Nova lokacija se preuzima ukoliko je promena u distanci veća od 1km ili ukoliko prođe 1 minut
    locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 60000, 1000, locationListener);
}

```

Slika 17. Provera permisija

4.2. Crtanje ruta

RouteFragment sadrži polja za unos željene početne i krajnje adrese. Nakon unosa, na mapi se iscrtava ruta koja spaja ove dve destinacije (slika 18).



Slika 18. Iscrtavanje rute na osnovu polazne i krajnje adrese

Kao i u primeru prikaza trenutne lokacije, i ovde je potrebno inicijalizovati *OSMDroid*. Klikom na dugme preuzimamo adrese koje korisnik unese i prosleđujemo funkciji *fetchRoute()*. Ove adrese potrebno je pretvoriti u geografske koordinate (širina i dužina) kako bi se mogle koristiti za iscrtavanje na mapi. Ovaj proces se zove geokodiranje i vršimo ga pomoću *Nominatim API*-ja. Kako ne bismo blokirali glavnu nit dok se šalju i preuzimaju odgovori zahteva, geokodiranje i dobavljanje tačaka rute odvajamo u zasebnu nit (slika 19).

```
private void fetchRoute(String startAddress, String endAddress) {
    executorService.execute(() -> {
        try {
            GeoPoint startPoint = getCoordinatesFromAddress(startAddress);
            if (startPoint == null) {
                postToast("Invalid start address");
                return;
            }

            GeoPoint endPoint = getCoordinatesFromAddress(endAddress);
            if (endPoint == null) {
                postToast("Invalid end address");
                return;
            }

            List<GeoPoint> routePoints = getRoute(startPoint, endPoint);
            if (routePoints == null || routePoints.isEmpty()) {
                postToast("Route not found");
                return;
            }

            if (!isAdded()) {
                return;
            }

            requireActivity().runOnUiThread(() -> drawRoute(routePoints));
        } catch (Exception e) {
            Log.e(tag, "RouteFragment", msg, "Error fetching route", e);
        }
    });
}
```

Slika 19. Odvajanje procesa geokodiranja i preuzimanja ruta od glavne niti

Metoda *getCoordinatesFromAddress()* kao parametar prima adresu koju korisnik unese. Adresa se ugrađuje u *URL* zahtev *Nominatim API*-ja. Kao odgovor očekujemo *JSON* objekat koji sadrži geografsku širinu i dužinu prosleđene adrese (slika 20) u vidu *GeoPoint* objekta. Vodite računa

da implementirate kod u slučaju kada zahtev nije dobro poslat ili kada adresa nije ispravna. Nakon dobavljanja geokodirane adrese, pozivamo metodu *getRoute()* koja vraća sve *GeoPoint* objekte od početne do krajnje tačke. Implementaciju ove metode možete pogledati u primeru projekta. Ove tačke prosleđujemo metodi *drawRoute()* kojom iscrtavamo rutu na mapi (slika 21).

```
private GeoPoint getCoordinatesFromAddress(String address) throws IOException, JSONException {

    String encodedAddress = URLEncoder.encode(address, "UTF-8");
    String url = "https://nominatim.openstreetmap.org/search?format=json&q=" + encodedAddress;
    Request request = new Request.Builder()
        .url(url)
        .header("User-Agent", "OSMDroidExample/1.0")
        .build();

    try (Response response = okHttpClient.newCall(request).execute()) {
        if (!response.isSuccessful() || response.body() == null) {
            return null;
        }
        JSONArray jsonArray = new JSONArray(response.body().string());
        if (jsonArray.length() == 0) {
            return null;
        }
        JSONObject object = jsonArray.getJSONObject(0);
        double lat = object.getDouble("lat");
        double lon = object.getDouble("lon");

        return new GeoPoint(lat, lon);
    }
}
```

Slika 20. Geokodiranje

Iscrtavanje rute se vrši pomoću *Polyline* klase. Predstavlja listu uzastopnih geokoordinata koje smo dobili u prethodnom koraku. *Polyline* klasa omogućava postavku parametara poput debljine i boje linije, kao i tipa linije za iscrtavanje. Objekat *Polyline* klase prosleđujemo *mapView* objektu i pomoću metode *invalidate()* ažuriramo prikaz.

```

private void drawRoute(List<GeoPoint> routePoints) {
    if (currentRouteLine != null) {
        mapView.getOverlays().remove(currentRouteLine);
    }
    currentRouteLine = new Polyline();
    currentRouteLine.setPoints(routePoints);
    currentRouteLine.setColor(0xFF0000FF);
    currentRouteLine.setWidth(10.0f);
    mapView.getOverlays().add(currentRouteLine);
    mapView.invalidate();
    IMapController controller = mapView.getController();
    controller.setZoom(15.0);
    controller.setCenter(routePoints.get(0));
}

```

Slika 21. Iscrtavanje rute na mapi

5. Primer

Domaći se nalazi na *Canvas-u* (canvas.ftn.uns.ac.rs) na putanji *Вежбе/08 Вежбе/08 Задамак.pdf*.

Primer *Vezbe8* možete preuzeti na sledećem linku:

<https://gitlab.com/mobilne-aplikacije/mobilne-aplikacije/>

Za dodatna pitanja možete se obratiti asistentima:

- Milan Jovkić (milan.jovkic@uns.ac.rs)
- Jelena Matković (matkovic.jelena@uns.ac.rs)